Sample Technical Documentation for clinical_shap.py using the proposed framework

Clinical Diagnosis Support System (SHAP-RandomForest)

Legislative Reference: Regulation (EU) 2024/1689 (AI Act), Annex IV
System Version: v1.5

1. General Description of the System

System Name: Clinical Diagnosis Support (Breast Cancer)

Provider: Demonstrative implementation for research purposes.

Intended Purpose: The system performs binary classification (malignant vs. benign) using structured clinical features derived from the Wisconsin Breast Cancer dataset.

System Output:

- Predicted class label
- SHAP-based explanation

The complete system architecture of the system is formally represented in Figure 1:

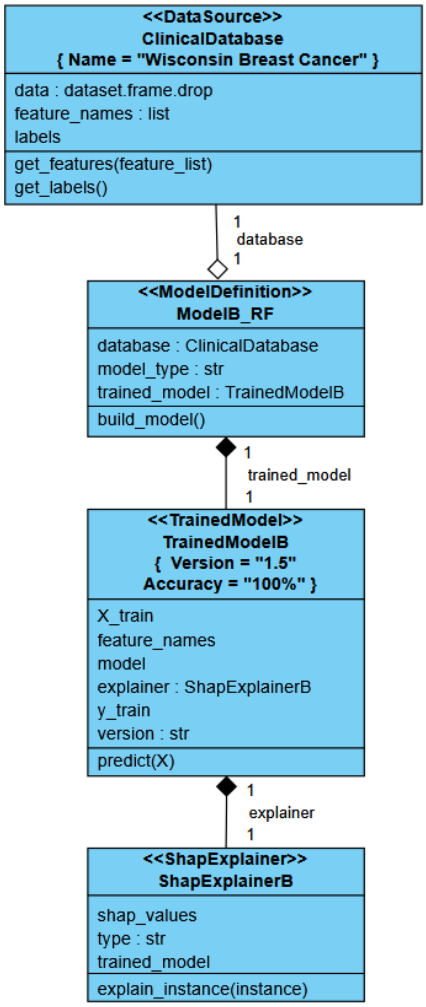

Figure 1 - Structural architecture of the Clinical Diagnosis Support system (UML with XAI stereotypes).

2. Detailed Description of System Elements and Development Process

The system is composed of four explicit modules, each represented in Figure 1:

<<DataSource>> ClinicalDatabase
Responsible for loading and providing access to the dataset, including:

- Data
- feature_names
- labels
- get_features(feature_list)
- get_labels()

<<ModelDefinition>> ModelB_RF
Responsible for:

- Model instantiation (RandomForest)
- Training process
- Construction of the trained model object
- build_model()

<<TrainedModel>> TrainedModelB
Represents the operational RandomForest model and includes:

- X_train
- y_train
- feature_names
- model
- explainer
- version
- predict(X)

Declared properties shown in Figure 1:

- Version = "1.5"
- Accuracy = "100%"

<<ShapExplainer>> ShapExplainerB
Responsible for generating post-hoc explanations, including:

- shap_values
- type
- trained_model
- explain_instance(instance)

All relationships between modules (composition and aggregation) are explicitly represented in Figure 1.

3. Data Requirements

Dataset: Wisconsin Breast Cancer (Diagnostic)

Number of Instances: 569

Number of Features: 30 numerical atributes

Target: Binary classification (malignant / benign)

The dataset and its name are structurally encapsulated within the <<DataSource>> component as represented in Figure 1.

No additional preprocessing, normalization, or transformation steps are implemented beyond the separation of features and labels.

4. Human Oversight and Monitoring

The system provides interpretability through the <<ShapExplainer>> module.
Human review occurs externally via inspection of:

- Predicted label
- SHAP waterfall visualization

No automated oversight, monitoring, or escalation mechanisms are implemented.

The explanation mechanism and its structural linkage to the trained model are explicitly represented in Figure 1.

5. Performance Metrics

The primary performance metric implemented is:

- Accuracy

The measured performance value is declared in the <<TrainedModel>> component and displayed in Figure 1:

- Accuracy = 100%

No additional performance metrics are computed in the system.

All performance declarations are structurally bound to the trained model element in Figure 1.

6. Risk Management Measures

No explicit risk management mechanisms are implemented within the system.

Structural decomposition is represented through the modular architecture shown in Figure 1, where Data source, Model definition, Trained model, and Explanation module are explicitly separated and traceable.

No additional mitigation mechanisms are implemented beyond structural separation.

7. Structural Representation and Traceability

The complete system structure, module responsibilities, and inter-module relationships are formally and exhaustively represented in Figure 1.

Figure 1 provides:

- Component boundaries
- Attribute visibility
- Method exposure
- Explicit multiplicities
- Composition and aggregation relationships
- Declared system properties (Version and Accuracy)

No additional components or processes beyond those represented in Figure 1 are implemented in the referenced system.